Functions I — Abstraction

Lecture 8

CS 1001

Kirk Duran

Today

- Reconnect expressions and assignment with a data-first execution model.
- Use trace tables to make changing name-to-value bindings explicit.
- Recognize repeated calculations as opportunities for abstraction.
- Define functions as named, parameterized computations.
- Distinguish definitions, calls, parameters, and arguments precisely.
- Perform a first lightweight trace of a simple function call.

Key idea

Today we move from *writing a calculation* to *naming a reusable calculation*.

Part 1: From Data to Program State

Start with the Data

- Programs operate on values: integers, floating-point values, strings, Booleans, and other data.
- Expressions inspect or combine existing values to produce new values.
- Assignment associates a name with the value produced by an expression.

Example

```
length = 3
width = 4
area = length * width
```

Image Placeholder

`data-first-model.png`

Assignment Evaluates First, Then Binds

Statement

```
area = length * width
```

1. Retrieve the values currently associated with `length` and `width`.
2. Evaluate `length * width`.
3. Produce one resulting value.
4. Bind the name `area` to that value.

Example

If `length -> 3` and `width -> 4`, then the right-hand side evaluates to `12`, after which `area -> 12`.

Trace Tables Make State Changes Explicit

Step	Statement	Result	Bindings afterward
1	`x = 4`	4	`x -> 4`
2	`y = x + 2`	6	`x -> 4, y -> 6`
3	`x = y * 3`	18	`x -> 18, y -> 6`

Definition

A **trace table** records selected execution steps and the program state resulting from each step.

Key idea

Tracing is prediction: determine the next value and state *before* asking Python to confirm it.

Practice: Trace a Short Sequence

Program

```
a = 5
b = a + 3
c = b * 2
a = c - 4
```

In class

Construct a four-row trace table. For each statement, record the right-hand-side value and all bindings after the statement executes.

Checkpoint

At the end, what are the values associated with `a`, `b`, and `c`? Which binding changed after it had already been created?

Part 2: Repetition Creates the Need for Abstraction

Three Cubes Repeat the Same Computational Structure

Repeated calculations

```
volume1 = length1 * width1 * height1
volume2 = length2 * width2 * height2
volume3 = length3 * width3 * height3
```

- The variable names differ because each cube has different data.
- The multiplication pattern is identical.
- Copying the formula preserves correctness only if every copy remains consistent.

In class

Circle the syntactic structure that is identical across all three statements. Underline the data that varies.

Scaling Repetition Is a Design Problem

- For 100 cubes, copying the same formula 100 times is technically possible.
- But repetition increases maintenance cost and the opportunity for inconsistent edits.
- More importantly, the program fails to express that all 100 calculations are instances of the *same idea*.

Design signal

When only the data changes while the computational rule remains fixed, the repeated rule is a candidate for abstraction.

Separate What Varies from What Remains Fixed

Varies between cases

```
length
width
height
```

- These are the input data.
- Each cube supplies different values.

Remains invariant

```
length * width * height
```

- This is the computational rule.
- Every cube uses the same rule.

Abstraction Names the Common Structure

Definition

Abstraction identifies a common computational idea and gives it an interface that can be reused without rewriting the underlying computation.

From repeated formula to named computation

`length * width * height` $\Longrightarrow$ `volume(length, width, height)`

Key idea

Good abstraction is not merely shorter code: it makes the program's conceptual structure explicit.

An Abstraction Has an Interface and an Implementation

Interface

- function name
- required inputs
- conceptual result

Example

```
volume(length, width, height)
```

Implementation

- statements inside the function
- expressions used to compute the result

Example

```
length * width * height
```

Key idea

The interface answers “how do I use it?”; the implementation answers “how is it computed?”

Part 3: Functions as Parameterized Computations

A Function Is a Named, Parameterized Computation

Definition

A **function** is a named computation whose parameters represent input values that may vary from one invocation to another.

Python

```
def volume(length, width, height):
  return length * width * height
```

- `volume` names the abstraction.
- The three parameters represent the varying dimensions.
- The body encodes the common rule.

Image Placeholder

`function-abstraction.png`

Mathematical Functions Provide a Useful Starting Analogy

Mathematics

$$f(x) = x^2$$

$$f(3) = 9$$

Python

```
def square(x):
  return x * x

square(3)
```

- A symbolic input stands for values supplied later.
- The same rule can be applied to many concrete inputs.
- Python functions additionally have execution semantics that we will formalize in Lecture 9.

Anatomy of a Python Function Definition

Definition

```
def volume(length, width, height):
```

Component	Role
`def`	begins a function definition
`volume`	function name
`length, width, height`	parameter names
`:`	begins the indented function body

Syntax versus semantics

The syntax tells Python how the function is written; the abstraction tells the programmer what computation the function represents.

The Function Body Encodes the Reusable Rule

Definition

```
def volume(length, width, height):
  result = length * width * height
  return result
```

- Indentation determines which statements belong to the function body.
- The body may contain multiple statements, not merely one expression.
- The body uses parameter names rather than hard-coded values so that the computation generalizes.

Example

For today, read `return result` as indicating the function's computed result. Lecture 9 will formalize the execution semantics of `return`.

Parameters Name the Roles of the Inputs

Definition

```
def rectangle_area(width, height):
  return width * height
```

- A parameter is a name used by the function definition to describe one input position.
- Parameter names should communicate the semantic role of the supplied value.
- Parameters are placeholders for values that will be supplied when the function is called.

Key idea

Parameters belong to the abstraction's interface: they state what information the computation requires.

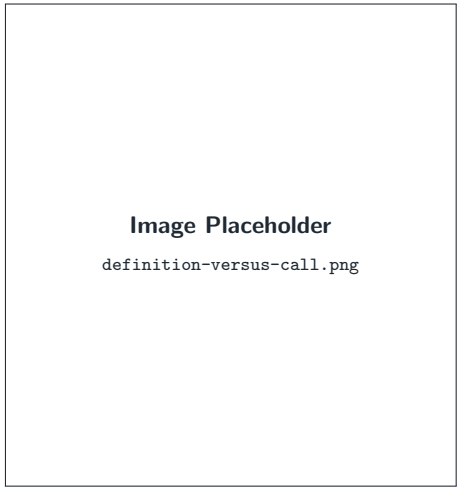
A Function Call Applies the Abstraction to Concrete Data

Definition

```
def volume(length, width, height):
  return length * width * height
```

Call

```
volume(2, 3, 4)
```



Key idea

One Definition Supports Many Calls

Function

```
def volume(length, width, height):
  return length * width * height
```

Call	Conceptual result
`volume(3, 4, 5)`	60
`volume(6, 7, 8)`	336
`volume(2, 3, 4)`	24

Key idea

Reuse comes from keeping the computation fixed while supplying different input values.

Part 4: Parameters, Arguments, and Calls

Parameters and Arguments Are Not Synonyms

Parameters

```
def volume(length, width, height):
```

- Names appearing in the definition.
- Describe the function's input positions.

Arguments

```
volume(2, 3, 4)
```

- Expressions appearing at the call site.
- Supply data for a particular invocation.

Terminology

Parameter: definition-side input name. Argument: call-side expression.

Positional Arguments Correspond by Position

Definition

```
def difference(first, second):
  return first - second
```

Call

```
difference(10, 4)
```

Correspondence

first argument 10 $\rightarrow$ `first`

second argument 4 $\rightarrow$ `second`

Key idea

For positional arguments, position determines which parameter receives which input value.

Changing Argument Order Can Change the Meaning

First call

```
difference(10, 4)

first -> 10
second -> 4

result: 6
```

Reversed call

```
difference(4, 10)

first -> 4
second -> 10

result: -6
```

Do not infer correspondence from names

Python does not inspect whether a caller's variable name resembles a parameter name when using positional arguments.

Caller Names and Parameter Names Can Differ Completely

Function

```
def rectangle_area(width, height):
  return width * height
```

Caller

```
w = 5
h = 3
result = rectangle_area(w, h)
```

`w -> 5 -> width` `h -> 3 -> height`

Key idea

The caller's names identify data in the caller; parameter names identify input roles inside the function abstraction.

Part 5: A First Function-Call Trace

A Simple Call Extends the Data-Flow Model

Program

```
def double(n):
  result = n * 2
  return result

x = 4
answer = double(x)
```

- Before the call, `x -> 4`.
- The call uses the current value represented by `x`.
- For this invocation, that value corresponds to parameter `n`.

Image Placeholder

`simple-call-data-flow.png`

The Parameter Represents the Supplied Input Value

Caller

`x -> 4`

↓ value supplied

Function invocation

`n -> 4`

```
result = n * 2
result -> 8
```

Key idea

The important invariant is the value `4`; `x` and `n` are different names used in different roles.

First Manual Function Trace

Stage	Relevant bindings / values
Before call	`x -> 4`
Input correspondence	`n -> 4`
Body calculation	`result -> 8`
After computation	`answer -> 8, x -> 4`

- This trace intentionally suppresses detailed call mechanics.
- It is sufficient to track how input data enters the abstraction and how the resulting value is used by the caller.

Next lecture

We will replace this simplified trace with the precise execution model for argument evaluation, local bindings, `return`, and resuming the caller.

Before and After: The Abstraction Clarifies Intent

Repeated calculations

```
v1 = l1 * w1 * h1
v2 = l2 * w2 * h2
v3 = l3 * w3 * h3
```

- The common idea is implicit.

Abstracted computation

```
v1 = volume(l1, w1, h1)
v2 = volume(l2, w2, h2)
v3 = volume(l3, w3, h3)
```

- The operation `volume` is explicit.

Key idea

Abstraction improves code by naming the concept, not merely by reducing character count.

Practice: Design the Abstraction Before Writing the Function

Repeated computation

A program repeatedly computes the cost of purchasing some quantity of an item using

```
price * quantity
```

In class

Determine: (1) a descriptive function name, (2) the parameters, (3) the computation that remains invariant, and (4) two example calls with different data.

Design target

Your function interface should make the calculation understandable before the reader inspects its body.

Practice: Read a Function from Its Interface

Definition

```
def miles_to_kilometers(miles):
  return miles * 1.60934
```

In class

Without tracing every operation, answer: What data does this function require? What conceptual transformation does it represent? What would `miles_to_kilometers(10)` mean?

Example

Then switch perspectives and identify the parameter name, argument value, and computation used by the implementation.

Part 6: The Mental Model Going Forward

The Prediction-First Workflow

$$\text{Predict} \longrightarrow \text{Trace} \longrightarrow \text{Execute} \longrightarrow \text{Compare}$$

- **Predict:** state what values and bindings you expect.
- **Trace:** justify the prediction one evaluation step at a time.
- **Execute:** use Python or the debugger as an observation tool.
- **Compare:** revise the mental model if observation disagrees with prediction.

Key idea

The debugger is most useful after you have a prediction to test.

What We Are Intentionally Deferring

- The exact runtime sequence when a function call begins and ends.
- The precise semantics of `return` as part of expression evaluation.
- Local execution environments and the lifetime of local bindings.
- Functions calling other functions and nested function-call expressions.
- Scope, multiple active calls, and the call stack.

Why defer these?

First establish the abstraction: a named computation applied to input data. Then study the execution machinery that makes the abstraction work.

Bridge to Lecture 9

Today

```
answer = double(x)
```

- We can already identify the abstraction, argument, parameter, and expected result.
- We have not yet fully explained what Python does between encountering the call and binding `answer`.

In class

What sequence of events must occur for `double(x)` to produce a value that assignment can bind to `answer`?

Key idea

Lecture 9 answers that question by treating a value-returning function call as an expression with precise execution semantics.

Takeaways

- Expressions evaluate to values; assignment binds names to resulting values.
- Trace tables expose how those bindings change as statements execute.
- Repeated computational structure is a signal that an abstraction may be useful.
- A function names a reusable computation; parameters represent input roles that may vary.
- Defining a function and calling a function are distinct operations.
- Parameters appear in definitions; arguments appear at call sites and supply concrete data.
- A first function trace follows values from caller data into parameterized computation and back to the caller's result.

Key idea

Mental model: **data** $\rightarrow$ **expressions** $\rightarrow$ **bindings** $\rightarrow$ **abstraction** $\rightarrow$ **reusable computation**.